CineSaga Microservice Project Synopsis

Project Synopsis

Title

CineSaga: Online Cinema Ticket Booking System Based on Microservices

Project Overview

CineSaga is a full-stack cinema ticket booking platform designed to simplify the process of discovering movies, reviewing details, selecting showtimes, booking seats, and completing ticket purchases online. The project is built as a microservice-based web application, with a Java and Spring Boot backend and a React-based frontend. It follows a distributed architecture so that major concerns such as user authentication, movie management, ticket booking, email notifications, service discovery, and request routing are separated into focused services.

The main idea behind the project is to simulate a real-world online cinema reservation system that is easy to use for customers, secure for administrators, and scalable for future expansion. Users can browse currently running movies and upcoming movies, open the movie detail page, read the synopsis, view cast and release details, choose the city and theater, select seats, and complete payment. After a successful transaction, the booking details are forwarded to the email service so that the customer receives a confirmation message automatically. In addition, authenticated users can leave comments on movie detail pages, while administrative users can add movies, actors, directors, and other cinema-related data.

The project demonstrates how a modern web system can be built using a service-oriented design. Instead of placing all logic inside one monolithic application, CineSaga divides responsibilities across multiple services, each with its own technology choices and business boundaries. This makes the system easier to maintain, easier to scale in parts, and easier to reason about during development.

Abstract

CineSaga is an end-to-end cinema ticket sale application that combines movie browsing, user registration and login, admin-controlled content management, seat selection, payment handling, and automated email delivery. The system uses Spring Cloud infrastructure for service discovery and gateway routing, Spring Security and JWT for authorization, PostgreSQL and MongoDB for data persistence, Kafka for asynchronous communication, Redis for caching, and React with Redux for the frontend experience. It is designed to reflect how a production-style online ticketing platform can be organized with microservices.

At the core of the project is a customer flow that begins with browsing movies and ends with a successful booking confirmation. The user journey is intentionally simple from the outside, but internally it relies on a distributed set of services communicating through synchronous and asynchronous mechanisms. This separation allows the system to keep responsibilities clean: user identity is handled by the user service, movie and booking operations are managed by the movie service, email

notifications are delegated to the email service, request orchestration is handled by the API gateway, and service registration is managed by the Eureka server.

The project is also valuable as a learning implementation because it touches many practical software engineering concerns at once: layered architecture, microservices, containerized infrastructure, state management on the frontend, security, caching, observability, and fault tolerance. Because of that, CineSaga is not only a cinema ticket booking application but also a complete demonstration of a contemporary full-stack microservice solution.

Project Objective

The primary objective of CineSaga is to provide a convenient digital platform for cinema ticket booking while ensuring that the system remains modular, secure, and extensible. The project aims to eliminate the inconvenience of manual or fragmented ticket reservation by allowing users to discover movies, inspect details, choose seats, and complete payment in one flow.

From a functional point of view, the system is intended to:

1. Allow visitors to browse movies that are currently running or will be released soon.
2. Display detailed movie information such as synopsis, cast, release date, and related cinema options.
3. Enable registered users to authenticate securely and interact with the platform.
4. Allow customers to choose city, theater, ticket count, seat arrangement, and payment details.
5. Send a confirmation email after successful ticket purchase.
6. Support user comments on movie pages for logged-in customers.
7. Restrict administrative actions such as adding movies, actors, directors, and related content to authorized admins only.
8. Maintain service separation so that changes in one feature area do not destabilize the whole system.

From a technical point of view, the project is built to demonstrate:

1. A microservice architecture using Spring Cloud.
2. Service discovery through Netflix Eureka.
3. Secure request routing through an API gateway.
4. JWT-based authentication and authorization.
5. Synchronous communication with WebClient and asynchronous communication with Kafka.
6. Persistent storage across different databases based on service needs.
7. Caching for frequently requested movie data.
8. Distributed tracing and monitoring with Micrometer Tracing and Zipkin.
9. A modern React frontend with Redux-based state management.

In short, the objective is both practical and architectural: the system solves a real user problem while also serving as a demonstration of how a distributed cinema booking platform can be implemented using current enterprise web technologies.

Problem Statement

Traditional cinema booking workflows can become inconvenient when ticket discovery, user authentication, booking, and confirmation are spread across disconnected systems or handled with weak coordination. Users expect fast browsing, reliable booking, and immediate confirmation, while administrators need controlled access to movie management tools. At the same time, a system like this should be resilient enough to support different service boundaries without becoming a maintenance burden.

CineSaga addresses these problems by splitting the system into microservices and assigning each business domain to a dedicated module. This approach reduces coupling, allows clearer ownership of functionality, and makes it easier to scale or modify individual parts of the application later. The problem is therefore not only to sell cinema tickets online, but also to build the platform in a way that is operationally organized and technically maintainable.

Scope of the Project

The project covers the main workflow of a cinema ticketing website from browsing to booking confirmation. Its scope includes:

- Movie discovery and detail viewing.
- User signup, login, and role-based access control.
- Admin-side content management for movies and related entities.
- Seat and ticket selection.
- Payment flow and success handling.
- Email notification after purchase.
- Communication between multiple backend services.
- Frontend routing and user interface for the booking experience.

The project does not attempt to cover every possible cinema business process. For example, it is focused on the booking journey and administrative content management rather than advanced business rules such as loyalty programs, dynamic pricing engines, recommendation systems, or third-party payment gateway integrations. That makes the current scope suitable for a semester project while still being rich enough to demonstrate a full-stack distributed architecture.

System Architecture

CineSaga follows a microservice architecture where the application is divided into independently deployed services. The main components are:

- **Eureka Server** for service registration and discovery.
- **API Gateway** for routing client requests and centralizing entry into the backend.
- **Movie Service** for movie catalogs, booking-related workflows, movie metadata, and related business operations.
- **User Service** for authentication, authorization, and user profile handling.
- **Email Service** for handling booking confirmation emails.
- **Frontend** for the React-based client interface.

This architecture gives the project a clear separation of concerns. The frontend talks primarily to the gateway, and the gateway forwards the request to the correct backend service. Services can discover each other through Eureka instead of relying on hard-coded addresses, which is important in a microservice environment. Some operations are handled synchronously, such as checking whether a user has the correct role before allowing an action. Other operations are handled asynchronously, such as sending ticket confirmation details to the email service through Kafka.

The backend services are organized in a layered style. Controllers expose REST endpoints, business layers contain the core logic, and data-access layers interact with databases. This structure keeps each service understandable and supports future expansion.

Major Modules and Their Roles

Module	Main Responsibility	Key Technologies
API Gateway	Routes client requests to backend services	Spring Cloud Gateway, Spring Security, Eureka Client
Eureka Server	Registers and discovers services	Netflix Eureka Server, Spring Security
User Service	Manages registration, login, JWT, and roles	Spring Web, Spring Security, MongoDB, JWT
Movie Service	Manages movie data, booking flow, comments, and payment-related logic	Spring Web, Spring Data JPA, PostgreSQL, Redis, WebFlux, Kafka, Resilience4j
Email Service	Sends booking confirmation emails	Spring Web, Kafka, Java Mail Sender, FreeMarker
Frontend	Provides browser-based user interface	React, Redux, Axios, Bootstrap

Technology Stack

Backend Technologies

The backend is implemented in Java 21 with Spring Boot and Spring Cloud. Maven is used for build and dependency management. The chosen backend stack is strong because it supports service-oriented development, security, data access, messaging, and observability in a consistent ecosystem.

Java 21 LTS

Java 21 is the main programming language for all backend services. It provides a modern, stable runtime with long-term support, making it suitable for a project that combines several services and libraries.

Spring Boot 4.0.2

Spring Boot is used to build each service quickly and cleanly with minimal boilerplate. It provides the application bootstrap, embedded server support, dependency management, and a simple model for building RESTful services.

Spring Cloud 2025.1.1

Spring Cloud provides the infrastructure needed for a distributed system. It supports service discovery, gateway routing, and resilient service communication across the microservices.

Spring Web and WebFlux

Spring Web is used for standard REST APIs. WebFlux is used where reactive and non-blocking communication is beneficial, especially between services that exchange data without blocking the request pipeline unnecessarily.

Spring Data JPA

Spring Data JPA is used in the movie service to simplify database access and repository implementation with PostgreSQL.

Spring Security and JWT

Security is a major concern in the application. Spring Security handles authentication and authorization, while JWT is used to generate and validate access tokens. This allows the system to protect admin-only operations and keep user actions tied to the correct identity and role.

Apache Kafka

Kafka is used for asynchronous service communication. It is especially useful in the booking confirmation flow, where the movie service publishes ticket information and the email service consumes the message to send a confirmation email.

Redis

Redis is used as a caching layer for frequently requested movie-related data. This improves response times for commonly accessed content such as movie listings and display pages.

PostgreSQL

PostgreSQL stores relational data for the movie service, including entities such as movies, actors, directors, categories, cities, schedules, and other structured booking data.

MongoDB

MongoDB stores user-related data in the user service. This choice fits well with flexible user documents and authentication-related records.

Java Mail Sender and FreeMarker

The email service uses Java Mail Sender to deliver emails and FreeMarker templates to format the message content in a readable and reusable way.

Resilience4j

Resilience4j provides circuit breaker support for service communication. This is important in a distributed setup because one service failure should not necessarily bring down the entire system.

Micrometer Tracing and Zipkin

These tools are used to trace requests across services and observe how a single user action moves through the distributed architecture.

Docker

Docker is used to run infrastructure components such as PostgreSQL, MongoDB, Kafka, Redis, and Zipkin in containers through the `docker-compose.yml` setup.

Frontend Technologies

The frontend is built with modern JavaScript and React. It is focused on usability, routing, state management, and smooth integration with backend APIs.

React 18

React is used to build the user interface as a component-based application. It allows the frontend to remain modular and responsive.

Redux

Redux manages shared application state, especially user state and movie-related data that needs to be available across multiple pages.

React Router

Routing is handled on the client side so users can navigate between the main page, detail page, ticket booking page, payment success page, and admin sections.

Axios

Axios is used to send HTTP requests to backend services through the gateway.

Bootstrap and CSS

These are used to shape the interface and keep the design responsive and accessible across screen sizes.

Formik, Yup, and Cleave.js

These libraries support form handling, validation, and formatted input fields such as payment-related inputs.

Swiper and React Toastify

Swiper supports carousel-like UI patterns, while React Toastify is used for user feedback and notifications.

Detailed Functional Description

User Journey

The user experience starts on the main page, where available movies are displayed. From there, the user can open the detail page of a movie and inspect its synopsis, actors, and other details. On the same page, the user can choose city and theater-related options, then proceed to the ticket buying page. During ticket booking, the user can select ticket count, ticket type such as student or adult, and chair selection. Once payment information is entered and the purchase is completed successfully, the system forwards the ticket details to the email service, which sends a confirmation email to the address provided by the customer.

This flow is designed to feel simple to the user, but it is supported by multiple backend operations working together behind the scenes.

Authentication and Authorization

The user service is responsible for login, registration, token generation, and role verification. Users are authenticated with Spring Security, and JWT is used to issue a secure token after successful login. This token is then used by other services to verify permissions. The application distinguishes between customers and administrators. Customers can browse, book, and comment, while administrators can access content-management functions such as adding movies or related entities.

Movie Management

The movie service is the core business service of the project. It stores and manages cinema-related data and coordinates booking-related functionality. It handles movie records, actor and director information, categories, cities, showtimes, comments, and payment-related ticket workflows. Because the service deals with the most user-visible data, it also uses caching for better performance and resilience mechanisms to improve stability.

Email Confirmation

The email service receives booking details asynchronously through Kafka. Once the message arrives, the service fills a FreeMarker template with the relevant ticket information and sends the email using Java Mail Sender. This design keeps email delivery separate from the booking transaction itself and makes the system more reliable and loosely coupled.

Gateway and Service Discovery

The API gateway acts as the main access point for client requests. Instead of allowing the frontend to talk directly to every service, the gateway receives requests, applies routing and security rules, and forwards them to the appropriate service. The Eureka server supports service registration so that the gateway and other services can find each other dynamically.

Architectural Benefits

This project architecture offers several advantages:

1. **Separation of concerns**: Each service has a focused responsibility.
2. **Maintainability**: Changes can be made inside one service with less impact on others.
3. **Scalability**: High-demand components can be scaled independently.
4. **Security**: Authentication and authorization are centralized and controlled.
5. **Performance**: Caching and non-blocking communication improve efficiency.
6. **Reliability**: Circuit breakers and asynchronous messaging reduce cascading failures.
7. **Observability**: Distributed tracing helps follow requests across services.

These qualities make the project a strong example of a production-inspired system design, even though it is built as an academic project.

Database Design and Persistence

The project uses different databases according to the nature of the data. PostgreSQL is used for movie-related relational data where structured relationships matter, such as movie records, cast

information, and booking-associated entities. MongoDB is used for user-related storage because it is flexible and works well with document-oriented user profiles and security-related metadata.

This decision reflects a practical architectural approach: the project does not force one database technology to do everything. Instead, it uses the database that best fits the service's data model. That is one of the strengths of microservice systems, because each service can choose a persistence strategy that suits its own needs.

Communication Between Services

The system combines synchronous and asynchronous communication styles.

Synchronous communication is used when a service needs an immediate response, such as checking whether a user has admin rights or whether a given action is allowed. For this, the project uses WebClient/WebFlux style service calls.

Asynchronous communication is used when the producer does not need to wait for the consumer to finish processing. The best example is the ticket confirmation email process. After payment, the movie service sends a message to Kafka, and the email service consumes it independently.

This mixture makes the system efficient and realistic. It also introduces the learner to one of the most important topics in microservice engineering: knowing when to ask for a direct response and when to publish an event instead.

Frontend Structure

The frontend is organized into reusable pages, layouts, services, reducers, and utility components. The main app is wrapped with a Redux provider and React Router, which allows state sharing and navigation across the application. The visible user paths include the main page, movie detail page, ticket purchase page, payment success page, and admin pages.

The frontend service layer maps to backend domains such as users, movies, actors, directors, cities, comments, payments, and movie images. This keeps the UI code clean and gives the React application a clear integration layer. The Redux store handles shared data, while route protection is used to prevent unauthorized access to admin-only screens.

Security Considerations

Security is a central part of the project because the system includes user accounts, protected booking actions, and admin-only operations. The user service handles authentication and token generation. The API gateway and backend services use security rules to protect endpoints and enforce access control. JWT tokens make authorization portable across services, which is important in a microservice design.

The project also secures the Eureka server and gateway layer, which helps prevent unprotected access to internal service infrastructure. This is a meaningful design choice because service discovery and routing components should not be left open in a real deployment.

Observability and Monitoring

A distributed application becomes difficult to troubleshoot without tracing. CineSaga addresses this through Micrometer Tracing and Zipkin. These tools help follow a request as it moves from the frontend through the gateway and into the internal services. This is especially useful when diagnosing payment

flow issues, authentication failures, or email delivery delays.

Observability is not just a technical extra in this project. It is part of making the architecture understandable and supportable when multiple services are involved.

Deployment and Runtime Setup

The repository shows a setup that is intended to run both application services and infrastructure containers locally. Docker Compose is used for PostgreSQL, MongoDB, Kafka, Redis, and Zipkin. The backend services can then be started in the usual Spring Boot sequence, while the frontend runs as a React application. This local runtime setup is useful for development and demonstration because it approximates a real multi-service environment.

The project also includes build and environment guidance for Java 21, Maven 3.9.x, Node.js 22.x, and npm 10.x or newer. That makes the project compatible with a modern development workflow.

Testing and Maintainability

The codebase includes separate modules and test scaffolding, which is a strong sign of maintainability. Each service can be tested independently, and the modular structure keeps future work manageable. The layered architecture also makes it easier to locate code by responsibility: controllers handle HTTP requests, business classes handle domain logic, and repositories or data-access objects manage persistence.

The same maintainability principle appears on the frontend, where reusable services and store modules reduce duplication and keep API interaction organized.

Future Improvements

While the current project already covers the core cinema booking flow, it can be expanded further in the future. Possible improvements include:

- integration with a real payment gateway,
- seat availability locking and stronger concurrency handling,
- recommendation features based on user behavior,
- richer admin analytics,
- notification channels beyond email,
- enhanced role management,
- production-grade container orchestration,
- stronger validation and monitoring dashboards,
- and more comprehensive automated tests.

These additions would make the system closer to a fully production-ready commercial application.

Conclusion

CineSaga is a well-structured online cinema ticket booking project that demonstrates both practical functionality and modern software architecture. Its main purpose is to offer a smooth booking experience for customers while giving administrators controlled access to content management

functions. Technically, it showcases a microservice ecosystem built with Spring Boot, Spring Cloud, React, PostgreSQL, MongoDB, Kafka, Redis, JWT, Docker, and related supporting tools.

The strongest aspect of the project is that it combines user-facing convenience with architectural discipline. The application is not only about booking a movie ticket; it is also about showing how a real-world service platform can be decomposed, secured, observed, and scaled. For that reason, CineSaga works well as both a functional cinema booking system and an academic demonstration of distributed system design.